

Projet de fin de formation —

Spécialisation Data Scientist

Scoring & prédiction de prix immobilier — NYC Property Sales (CSV)

Version apprenant — Décembre 2025

Contexte

Vous rejoignez **MetroValuation**, une start-up qui fournit des outils d'aide à la décision aux professionnels de l'immobilier (fonds d'investissement, marchands de biens, banques). Votre équipe travaille sur le marché de New York (5 boroughs) et souhaite automatiser l'estimation d'une **valeur de vente attendue** pour chaque transaction enregistrée par la ville.

Le produit visé est un **score de valorisation** : à partir des caractéristiques d'un bien (localisation, type, surface, année de construction, etc.), le modèle prédit un prix « théorique » puis calcule un indicateur d'écart (surcote/décote) par rapport au prix observé. Ce score sert à : (1) détecter des ventes atypiques, (2) repérer des opportunités potentiellement sous-évaluées, et (3) prioriser des analyses humaines avec un budget limité.

Objectifs pédagogiques

À l'issue du projet, vous aurez (1) structuré un dépôt Git professionnel, (2) réalisé une EDA complète en Python, (3) construit une **pipeline de préparation** robuste (nettoyage, encodage, gestion des valeurs manquantes/outliers), et livré un modèle baseline, puis (4) un modèle finetuné avec comparaison rigoureuse et recommandations d'usage (métriques, erreurs, stratégie de scoring). La reproductibilité, la qualité de code et la capacité à relier la performance ML à une décision métier seront évaluées.

DATAGONG - SAS au capital de 1000€

Siège Social : 17 rue Paul Verlaine, 94410 Saint-Maurice

SIRET : 913 142 055 00015 - APE : 85.59A

Enregistré auprès de la Direction Régionale et Interdépartementale de l'Économie, de l'Emploi, du Travail et des Solidarités (DRIETS)
d'Île-de-France sous le numéro 11941135494

DATAGONG est un Organisme de Formation Certifié QUALIOPF au titre des actions de formation

Données et cible

Le jeu de données fourni contient des enregistrements de ventes immobilières à New York sur une période glissante d'environ 12 mois (un enregistrement par transaction). Il inclut typiquement : **borough**, **neighborhood**, classes de bâtiment, unités résidentielles/commerciales, surfaces (land / gross square feet), année de construction, code postal, adresse, **sale date** et **sale price**.

Variable cible : SALE PRICE (régression). Vous pouvez choisir de modéliser directement le prix ou sa transformation (ex. log) si cela améliore la stabilité.

Format : **CSV fourni par l'équipe pédagogique** (issu du dataset public Kaggle « NYC Property Sales »).

Contraintes pédagogiques (formation Data Scientist)

- 1 Pas de SQL : travail uniquement à partir du CSV.
- 2 Pas de Power BI : restitution via notebooks, rapport Markdown et figures Python.
- 3 Orientation modélisation prédictive / scoring : pipeline ML, métriques, interprétation et utilisation du score.

Étape 1 — Création du repo Git + EDA

Objectif : mettre en place un dépôt reproductible puis **comprendre la structure des ventes** (qualité des données, distributions, segments, facteurs associés au prix).

Travail attendu :

- 1 Initialiser le dépôt Git (README, .gitignore, structure) et réaliser au moins 3 commits propres (setup, EDA, baseline plan).
- 2 Charger le CSV et produire un mini dictionnaire de données (type, rôle, exemples).
- 3 Contrôler la qualité : valeurs manquantes, doublons, incohérences (surfaces = 0, années aberrantes), formats (dates, numériques).
- 4 Analyser la cible : distribution du SALE PRICE, effets de la transformation log, analyse des outliers (prix très faibles/0 ou très élevés).
- 5 Segmenter : comparaison par borough, neighborhood, building class, surfaces, année de construction, saisonnalité (sale date).
- 6 Définir le protocole d'évaluation (train/valid/test ou CV), choix de métriques et stratégie de traitement des valeurs extrêmes.

Étape 2 — Livrer le modèle Baseline (préparation + encodage)

Objectif : livrer un premier modèle de prédiction de prix robuste et reproductible, avec une pipeline de préparation complète. Le baseline doit être simple, explicable et servir de référence.

Attendus (techniques) :

- 1 Construire une pipeline scikit-learn : split, imputation, encodage des variables catégorielles (One-Hot), scaling des numériques si nécessaire.
- 2 Mettre en place une **ColumnTransformer** et une **Pipeline** pour éviter les fuites de données.
- 3 Choisir un baseline : **DummyRegressor** (référence) + un modèle simple (ex. Ridge/Lasso, ElasticNet).
- 4 Évaluer : MAE et RMSE (ou RMSE sur log-prix), R2, analyse des résidus (erreurs par segment : borough, building class).
- 5 Produire un fichier de scoring : id (ou index), y_{true} , y_{pred} , erreur absolue, et un **discount_score** = $y_{pred} - y_{obs}$ (ou équivalent).
- 6 Sauvegarder le modèle (joblib) et documenter la reproductibilité (seed, versions, commande d'exécution).

Livrable intermédiaire (Checkpoint 2) : modèle baseline + rapport de métriques + fichier de scoring.

Étape 3 — Livrer le modèle Finetuné (optimisation)

Objectif : améliorer la précision et l'utilité du score en documentant les gains, les risques et les compromis. Vous devez comparer baseline vs finetuné avec un protocole strictement identique.

Pistes d'amélioration possibles (à sélectionner et justifier) :

- 1 **Modèles** : RandomForestRegressor, GradientBoosting/HistGradientBoosting, XGBoost/LightGBM/CatBoost (si autorisé).
- 2 **Feature engineering** : âge du bâtiment (année courante - year built), surfaces normalisées, log(surfaces), indicateurs de valeurs manquantes, agrégats par neighborhood/zip (sur train uniquement).
- 3 **Gestion des outliers** : filtrage raisonné (transactions non comparables), winsorisation, objectif log, Huber/quantile loss.
- 4 **Optimisation** : RandomizedSearchCV / Optuna (si autorisé) + validation croisée, avec suivi du temps de calcul.
- 5 **Interprétabilité** : permutation importance, SHAP (optionnel), analyse des erreurs sur segments.
- 6 **Utilisation du score** : définir une stratégie « top K opportunités » (ex. 5% biens les plus décotés) et vérifier la stabilité par borough.

Livrable intermédiaire (Checkpoint 3) : modèle finetuné + rapport comparatif + scoring final.

Livrables et structure du dépôt Git

- 1 **Dépôt Git** complet (README, .gitignore, historique de commits lisible).
- 2 **Notebooks** : EDA (étape 1), baseline (étape 2), finetune (étape 3).
- 3 **Artefacts modèles** : baseline et finetuné (joblib), dossier models/.
- 4 **Scoring** : CSV final avec prédictions + discount_score + top opportunités (optionnel) + rapport (Markdown/PDF).

Organisation recommandée du dépôt :

```
/
|-- README.md
|-- data/
|   |-- raw/
|   |-- nyc_property_sales.csv
|-- notebooks/
|   |-- 01_setup_repo_et_eda.ipynb
|   |-- 02_baseline_model.ipynb
|   |-- 03_fineturned_model.ipynb
|-- src/
|   |-- data_prep.py
|   |-- train.py
|   |-- infer.py
|   |-- metrics.py
|-- models/
|   |-- baseline.joblib
|   |-- finetuned.joblib
|-- reports/
|   |-- figures/
|   |-- model_report.md
|-- requirements.txt
|-- .gitignore
```

README — contenu minimal :

1) Contexte métier et objectif du scoring. 2) Description des données + règles de nettoyage (prix nuls, outliers, dates). 3) Installation (env, dépendances). 4) Reproduire : EDA → baseline → finetune → scoring. 5) Résultats (MAE/RMSE/R2) + analyse d'erreurs. 6) Stratégie d'utilisation (top K% décotes) + limites et risques.

Grille d'évaluation (synthèse)

- 1 **Cadrage & traçabilité** : repo propre, hypothèses explicites, reproductibilité.
- 2 **EDA** : qualité des contrôles, analyses pertinentes, compréhension des segments.
- 3 **Pipeline ML** : preprocessing correct (pas de leakage), encodage/typage, validation rigoureuse.
- 4 **Baseline** : références claires (Dummy + modèle simple), métriques commentées.
- 5 **Finetuning** : amélioration démontrée, hyperparamètres/features justifiés, analyse des erreurs.
- 6 **Interprétabilité & usage** : facteurs clés, top opportunités, stabilité par borough.
- 7 **Qualité de code** : lisibilité, factorisation (src/), seeds, commentaires utiles.

Conseils pratiques

- 1 Commencez par fixer un ordre de grandeur : **DummyRegressor** puis Ridge sur **log(SALE PRICE)**.
- 2 Vérifiez les cas **SALE PRICE = 0** ou très bas (transferts/transactions atypiques) : documentez votre règle de traitement.
- 3 Ne mélangez pas train/test dans vos agrégats (moyennes par quartier) : calculez-les **sur le train uniquement**.
- 4 Analysez les erreurs par borough : un modèle performant globalement peut être mauvais sur un segment précis.
- 5 Pensez produit : comment utiliser le **discount_score** (top K%) et comment contrôler les faux positifs.